FatTummy Technical Architecture and Open-Source Usage Guide

Dwij Shukla

June 19, 2026

1 Executive Summary

FatTummy is a compact, declarative Python framework designed to unify hardware detection, dependency management, data ingestion, local and cloud inference adapters, a native mixture-of-experts model, and a thin fine-tuning trainer behind a single API. Its main product promise is convenience: users can move between cloud APIs, local model runners, and native model experimentation without changing the high-level workflow.

From a technical perspective, the project has a promising abstraction layer and a clear research-oriented direction, especially around its native MOOE model. However, several parts remain prototype-grade: the training loop is mostly scaffolding, the MOOE routing path is not yet performance-oriented, and runtime dependency installation creates reproducibility and operational risks. As an open-source package, its value is clearest as an easy-to-install experimentation framework: users can start with `pip install fattummy`, then explore model creation, fine-tuning, local inference, and API-backed chat through a small Python interface.

2 Practical Overview: Why FatTummy Feels Simple

FatTummy is designed around one main idea: a researcher should be able to describe what they want in a few Python calls, and the framework should handle the repetitive setup work. Instead of manually wiring together hardware checks, package installs, dataset loading, model selection, API clients, local runners, and chat loops, FatTummy exposes a small builder-style interface.

The simplest version is:

```
import FatTummy as ft

ft.build()
```

That single call launches the interactive wizard. The user chooses whether they want to make a model, fine-tune a model, or chat through an API. FatTummy then asks for only the required details and fills in reasonable defaults where possible.

2.1 Making Models

To make a native model, the user selects the model-building path and chooses the MOOE architecture. Programmatically, the workflow is intentionally short:

```
import FatTummy as ft

ft.build(interactive=False)
ft.modelbuild("10B")
ft.type(ft.MOOE)
ft.chat()
```

This is simple because the user does not need to manually instantiate configuration classes, create layers, wire routing modules, or prepare a chat loop. The framework stores the requested scale, resolves the architecture, initializes the runtime, builds the native MOOE model, and opens an interactive session.

The current native model path is best for experimentation, demos, and research exploration. For very large production models, the MOOE implementation still needs distributed routing, sharding, and memory optimization.

2.2 Training and Fine-Tuning

FatTummy also tries to make training feel declarative. A user provides the model, dataset, and number of epochs, then asks the engine to fine-tune:

```
import FatTummy as ft

ft.build(interactive=False)
ft.engine("hf")
ft.type("gpt2")
ft.data("my-org/my-dataset")
ft.finetune(epochs=3)
```

The framework handles the high-level steps: it records the Hugging Face model name, resolves the dataset source, checks whether data should be streamed or downloaded, selects GPU/CPU/TPU execution where possible, and passes the model and data into the trainer.

For researchers, the benefit is speed of setup. They can focus on model choice and data rather than boilerplate. However, the current trainer is still experimental: it needs a complete production training loop with real tokenization, batching, loss computation, checkpointing, evaluation, and logging.

2.3 Using Models Locally

FatTummy can also route prompts to local model backends. For Ollama, the user can point the engine at a local model name:

```
import FatTummy as ft

ft.build(interactive=False)
ft.engine("ollama")
ft.type("llama3")
ft.chat()
```

For Hugging Face models, the same general shape applies:

```
import FatTummy as ft

ft.build(interactive=False)
ft.engine("hf")
ft.type("gpt2")
ft.chat()
```

The simplicity comes from the shared interface. Whether the model is native, local through Ollama, or loaded from Hugging Face, the user still thinks in terms of engine, model type, data, temperature, and chat.

2.4 Using Cloud APIs

API usage is also reduced to a few calls. For example, an OpenAI-backed chat session can be configured as:

```
import FatTummy as ft

ft.build(interactive=False)
ft.engine("openai")
ft.key("YOUR_API_KEY")
ft.chat()
```

The same concept applies to Anthropic and Gemini by changing the engine name and key. The user does not need to directly import each vendor SDK, remember each provider's request format, or write a separate chat loop. FatTummy's cloud adapter layer hides those differences behind one `generate()` interface.

2.5 Simple Mental Model

The user-facing mental model is:

`build` → choose `engine` → choose `model/type` → add `data` if needed → `chat` or `finetune`

In practice, this means FatTummy turns several different workflows into one consistent pattern:

- **Make a model:** choose a scale and MOOE type, then chat.
- **Train or fine-tune:** choose a model, attach data, set epochs, then call `finetune()`.
- **Use a local model:** choose `hf` or `ollama`, set the model name, then chat.
- **Use a cloud API:** choose `openai`, `anthropic`, or `gemini`, provide a key, then chat.
- **Change behavior:** adjust temperature, token limits, datasets, and epochs with small chainable calls.

The result is a framework that feels simple because it compresses many backend-specific operations into a small number of verbs: `build`, `engine`, `type`, `data`, `temp`, `chat`, and `finetune`.

3 Technical Architecture

3.1 High-Level Design

After installation with `pip install fattummy`, the project follows a builder-style execution model centered on `FatTummyEngine`. Users compose a runtime context through chainable calls, then route work to one of three execution targets:

- **Cloud inference adapters:** wrappers around providers such as OpenAI, Anthropic, and Gemini.
- **Local inference adapters:** integrations for Hugging Face models and Ollama-backed local serving.
- **Native model path:** a custom MOOE model implementation for direct experimentation.

The intended data flow is straightforward:

`data()` → `data/loader.py` → trainer or inference adapter → generated output or fine-tuned model

The data loader decides whether to stream or download data, using a threshold-based approach for large datasets. This is useful for research workloads where corpora may be too large to fit comfortably on disk.

3.2 Core Project Files

- `README.md`: project introduction and user-facing overview.
- `pyproject.toml` and `setup.py`: packaging, metadata, and licensing information.
- `FatTummy/engine.py`: high-level builder API and execution orchestration.
- `FatTummy/models/mooe.py`: native MOOE model configuration and layers.
- `FatTummy/inference/cloud_adapters.py`: cloud provider integrations.
- `FatTummy/inference/local_adapters.py`: local Hugging Face and Ollama integrations.
- `FatTummy/data/loader.py`: dataset loading, streaming, and download decisions.
- `FatTummy/installer.py`: hardware-aware dependency and environment setup.
- `FatTummy/tuning/trainer.py`: fine-tuning orchestration and training scaffolding.

4 Native MOOE Model Analysis

4.1 Design Strengths

The native MOOE implementation provides a compact mixture-of-experts design. Its configuration object defines hidden sizes, intermediate sizes, expert counts, and parameter heuristics. It also appears compatible with the Hugging Face Transformers configuration pattern through a `PretrainedConfig`-style interface.

The core routing design is conceptually simple:

$$\text{gate logits} \rightarrow \text{softmax probabilities} \rightarrow \text{top-}k \text{ expert selection} \rightarrow \text{weighted expert dispatch} \rightarrow \text{re-aggregation}$$

This makes the implementation easy to inspect and modify, which is valuable for a research framework.

4.2 Technical Limitations

The current routing strategy is not yet suitable for large-scale training or inference. Important limitations include:

- **Naive routing:** token-to-expert dispatch appears to rely on simple synchronous control flow rather than optimized batched routing.
- **Memory risk:** large expert counts and hidden sizes can quickly create memory pressure without quantization, activation checkpointing, or sharding.

The MOOE path is therefore best understood as a research prototype rather than a production-scale MoE engine.

5 Inference and Training

5.1 Inference Adapters

The cloud adapters provide a useful abstraction over third-party vendor SDKs and convert provider errors into framework-level exceptions. This is good for API consistency, but production use would benefit from retries, backoff, timeout controls, structured logging, and provider-level rate-limit handling.

The local Hugging Face adapter uses convenience defaults such as automatic device mapping and half precision. This improves usability, but large-model workloads need more explicit controls for quantization, CPU/GPU offloading, sharded loading, and memory budgeting. The Ollama adapter adds practical local serving support by communicating with a local daemon and pulling models when needed.

5.2 Training Pipeline

The trainer acts as a thin orchestrator for GPU, CPU, and TPU paths. Its TPU fallback through `torch_xla` is directionally useful, but the current training logic is not production-ready. The GPU loop appears largely mock-oriented, with placeholder data and dummy inputs.

A production training pipeline should add:

- real dataset batching and tokenization;
- loss computation and evaluation metrics;
- checkpoint save and resume support;
- gradient accumulation and mixed precision;
- distributed optimizer support;
- validation loops and regression tests.

6 Code Walkthrough

This section reviews the actual packaged source from the FatTummy 0.1.5 source distribution. The implementation is small and intentionally direct: most user-facing behavior flows from the package-level singleton API into `FatTummyEngine`, which then selects a cloud adapter, local adapter, native model, or trainer.

6.1 Package Entry Point

The package-level `FatTummy/__init__.py` file exposes the framework as a singleton builder API. Calls such as `ft.build()`, `ft.engine()`, `ft.data()`, `ft.temp()`, `ft.chat()`, and `ft.finetune()` all delegate to one shared `FatTummyEngine` instance. This makes the public API easy to use in notebooks and short scripts, but it also means global mutable state is central to the design. For production pipelines, an explicit engine object is safer because it avoids hidden state across cells, tests, or long-running processes.

6.2 Engine Orchestration

`FatTummy/engine.py` is the main control layer. Its constructor initializes defaults for engine name, model scale, model type, data sources, API key, temperature, token limits, epochs, and Hugging Face token state. Builder methods update these fields and return `self`, enabling chains such as `ft.build(interactive=False).engine("openai").key(...).chat()`.

The private `_compile_and_initialize()` method is the key routing point. It first calls the installer unless the runtime is API-only. Then it chooses between cloud adapters, local adapters, or a native MOOE model. Generation goes through `generate()`, while terminal interaction goes through `chat()`. Fine-tuning calls `FatTummyTrainer` with the selected model, dataset list, and epoch count.

A notable implementation issue is that `type()` only validates the native MOOE target and otherwise accepts arbitrary architecture values. That flexibility helps adapter-based use cases, but stronger validation would improve error messages and reduce misconfiguration.

6.3 Interactive Wizard

`FatTummy/interactive.py` implements the terminal wizard. It offers Breeze and advanced modes, asks the user to choose between model creation, fine-tuning, and API chat, then applies the collected configuration to a `FatTummyEngine`. The wizard also resolves datasets before running the selected action.

This is a good onboarding layer because it hides complexity from first-time users. The main weakness is that prompting, configuration validation, environment setup, and action execution are tightly coupled. A cleaner design would separate these into: prompt collection, config schema validation, engine construction, and action execution. That would make the same configuration path reusable from CLI, notebooks, tests, and future web UIs.

6.4 Dataset Loading

`FatTummy/data/loader.py` supports local files and Hugging Face dataset repositories. It uses a 500 MB threshold to choose between full loading and streaming. Local files are routed by extension, including JSON, JSONL, CSV, and text. Hugging Face repositories are identified with a simple `namespace/name` heuristic.

The design is practical, but the loader should add clearer validation for malformed paths, missing files, unsupported extensions, and private repositories. It should also preserve dataset metadata such as source name, split, streaming mode, and size estimate so downstream training can report exactly what was used.

6.5 Cloud Inference Adapters

`FatTummy/inference/cloud_adapters.py` defines a base adapter plus OpenAI, Anthropic, and Gemini implementations. Each adapter accepts an API key, calls the provider SDK, and wraps failures in `FatTummyNetworkError`. The factory function `get_cloud_adapter()` maps an engine name to the appropriate adapter.

This adapter boundary is one of the strongest architectural choices in the project. It keeps vendor-specific SDK logic out of the engine. The next step is to make adapters production-grade by adding timeouts, retries, rate-limit handling, streaming output, provider-specific model configuration, and consistent token accounting.

6.6 Local Inference Adapters

The local adapter module supports Ollama and Hugging Face. The Ollama adapter shells out to `ollama list`, `ollama pull`, and `ollama run`. The Hugging Face adapter loads `AutoTokenizer` and `AutoModelForCausalLM`, uses `device_map="auto"`, and defaults to half precision.

The local adapters optimize for convenience, but they need stronger resource controls. Large models need explicit quantization options, maximum memory maps, CPU offload settings, context-length controls, and clear out-of-memory recovery paths. The Ollama adapter should also avoid indefinite subprocess hangs by enforcing timeouts.

6.7 Native MOOE Model

`FatTummy/models/mooe.py` contains `MOOEConfig`, `Expert`, `MOOELayer`, and `MOOE`. The model uses embeddings, a stack of MOOE layers, layer normalization, and an LM head. Inside each MOOE layer, a gate projects hidden states to expert logits, selects top-k experts, and combines expert outputs according to routing weights.

The implementation is readable and useful for experimentation. However, the current per-token expert dispatch style is the main scaling bottleneck. Production MoE systems normally batch tokens per expert, apply capacity constraints, add load-balancing losses, and distribute experts across devices. Without those pieces, the native MOOE path is best positioned as an educational or research prototype.

6.8 Trainer

`FatTummy/tuning/trainer.py` defines `FatTummyTrainer`. It detects whether `torch_xla` is available and chooses a TPU path or a GPU/CPU path. The TPU branch sets up XLA device handling and parallel loading scaffolding. The GPU/CPU branch creates an optimizer and runs a short placeholder loop.

This file is the largest gap between the framework's promise and its current implementation. A real fine-tuning trainer needs tokenized batches, dataloaders, loss computation, gradient accumulation, mixed precision, evaluation, checkpointing, resume support, logging, and deterministic test coverage. Until then, the fine-tuning feature should be described as experimental.

6.9 Installer and Environment Management

`FatTummy/installer.py` checks Python version compatibility, installs API dependencies, detects TPU or GPU availability, and installs hardware-specific packages. This creates a convenient first-run experience, especially in notebooks, but it is risky for reproducibility because runtime package installation can mutate the user's environment.

A production-ready approach would provide pinned dependency groups, lock files, container images, or extras such as `fattummy[api]`, `fattummy[hf]`, `fattummy[gpu]`, and `fattummy[tpu]`. The installer could still exist as a helper, but it should not be the primary dependency-management strategy for serious deployments.

6.10 Exception Model

The exceptions module defines framework-specific errors for network and out-of-memory failures. This is a good start because it prevents provider SDK exceptions from leaking directly to users. The exception hierarchy should be expanded to include configuration errors, dataset loading errors, dependency errors, authentication errors, and unsupported backend errors.

6.11 Code-Level Summary

Overall, the codebase is intentionally minimal and approachable. Its strongest parts are the public API shape, adapter boundary, dataset streaming decision, and native MOOE experimentation path. Its weakest parts are training completeness, runtime dependency mutation, lack of tests, lack of observability, and limited production controls for large-model inference. The right near-term goal is not to add more features, but to harden the existing control flow and make each advertised feature work reliably end to end.

7 Performance and Scaling

To scale FatTummy beyond prototype workloads, the most important technical investment is the MOOE execution path. The project should prioritize vectorized expert routing, expert batching, capacity-aware dispatch, and distributed expert placement. For inference throughput, adapters should support batching, streaming responses, async I/O, request queues, and rate limiting.

Large local models will also require quantization support, explicit memory planning, and reproducible deployment profiles. Useful additions include 8-bit or 4-bit loading, activation checkpointing, containerized runtime images, and benchmark scripts that measure latency, throughput, memory use, and cost.

8 Operational Risks

- **Runtime dependency installation:** automatic `pip` installs are convenient but risky for reproducibility and restricted environments.
- **Limited observability:** coarse warnings and prints should become structured logs, metrics, and traceable errors.
- **Network fragility:** cloud adapters need retry policies, timeout controls, and clearer error classification.
- **Testing gaps:** unit tests and integration tests are needed for routing, adapters, data loading, and trainer behavior.

9 Open-Source Qualities and Use Cases

Because FatTummy is open source and installable with `pip install fattummy`, its strongest value is accessibility. It gives researchers, students, and builders a low-friction way to experiment with model creation, fine-tuning workflows, local inference, and cloud API routing without first assembling a large custom codebase. Its long-term promise is that teams can move from quick experiments toward industry-grade LLM workflows with minimal code, using the same simple interface for model creation, training, fine-tuning, and deployment-oriented inference.

9.1 Key Open-Source Qualities

- **Easy installation:** users can begin with `pip install fattummy` and a short Python script.
- **Small API surface:** the main workflow is built around simple verbs such as `build`, `engine`, `type`, `data`, `chat`, and `finetune`.
- **Readable architecture:** the codebase is compact enough for contributors to understand the engine, adapters, trainer, and MOOE model without navigating a large framework.

- **Backend flexibility:** the same interface can route work to native MOOE models, local Hugging Face models, Ollama models, or cloud APIs.
- **Research-friendly design:** the native MOOE implementation is easy to inspect, modify, and extend for experimentation.
- **Minimal-code LLM development:** users can describe an LLM workflow in a few calls instead of writing separate scripts for setup, loading, training, inference, and API routing.
- **Industry-grade direction:** with stronger training, scaling, and deployment layers, the same simple interface can support serious LLM workflows while preserving ease of use.
- **Community extensibility:** contributors can add new adapters, dataset loaders, model variants, trainer improvements, and examples without changing the user-facing workflow.

9.2 Primary Use Cases

- **Fast AI prototyping:** quickly test an idea by choosing a backend, model, dataset, and chat or fine-tuning action.
- **Industry-grade LLM workflows:** build toward robust local, cloud, or native LLM systems while keeping the user code compact and readable.
- **Research experiments:** study mixture-of-experts behavior by modifying the native MOOE layers and routing logic.
- **Teaching and demos:** show students how model selection, data loading, local inference, and API-backed chat fit together in one framework.
- **Local model workflows:** use Hugging Face or Ollama models through a shared FatTummy interface instead of writing backend-specific scripts.
- **API-backed chat tools:** switch between OpenAI, Anthropic, and Gemini-style providers while keeping the same high-level application code.
- **Dataset experimentation:** connect local files or Hugging Face datasets and let the loader choose streaming or full loading based on size.

9.3 Who Benefits Most

FatTummy is especially useful for researchers who want quick experiments, students learning how AI tooling fits together, open-source contributors who want a compact codebase to improve, and builders who want a single lightweight interface for local models, native models, and API models. As the training and scaling layers mature, it can also serve teams that want industry-grade LLM capability without sacrificing the simplicity of a few high-level Python calls.

10 Conclusion

FatTummy has a useful open-source foundation: a unified API, multiple inference backends, data-loading convenience, and a native MOOE direction that could become a meaningful differentiator for researchers and builders. The strongest path forward is to keep installation simple with `pip install fattummy`, harden the trainer, optimize the MOOE implementation, improve reproducibility, and expand examples. If those issues are addressed, the project could become a credible research-to-production framework for hybrid local, cloud, and native expert-model workflows.